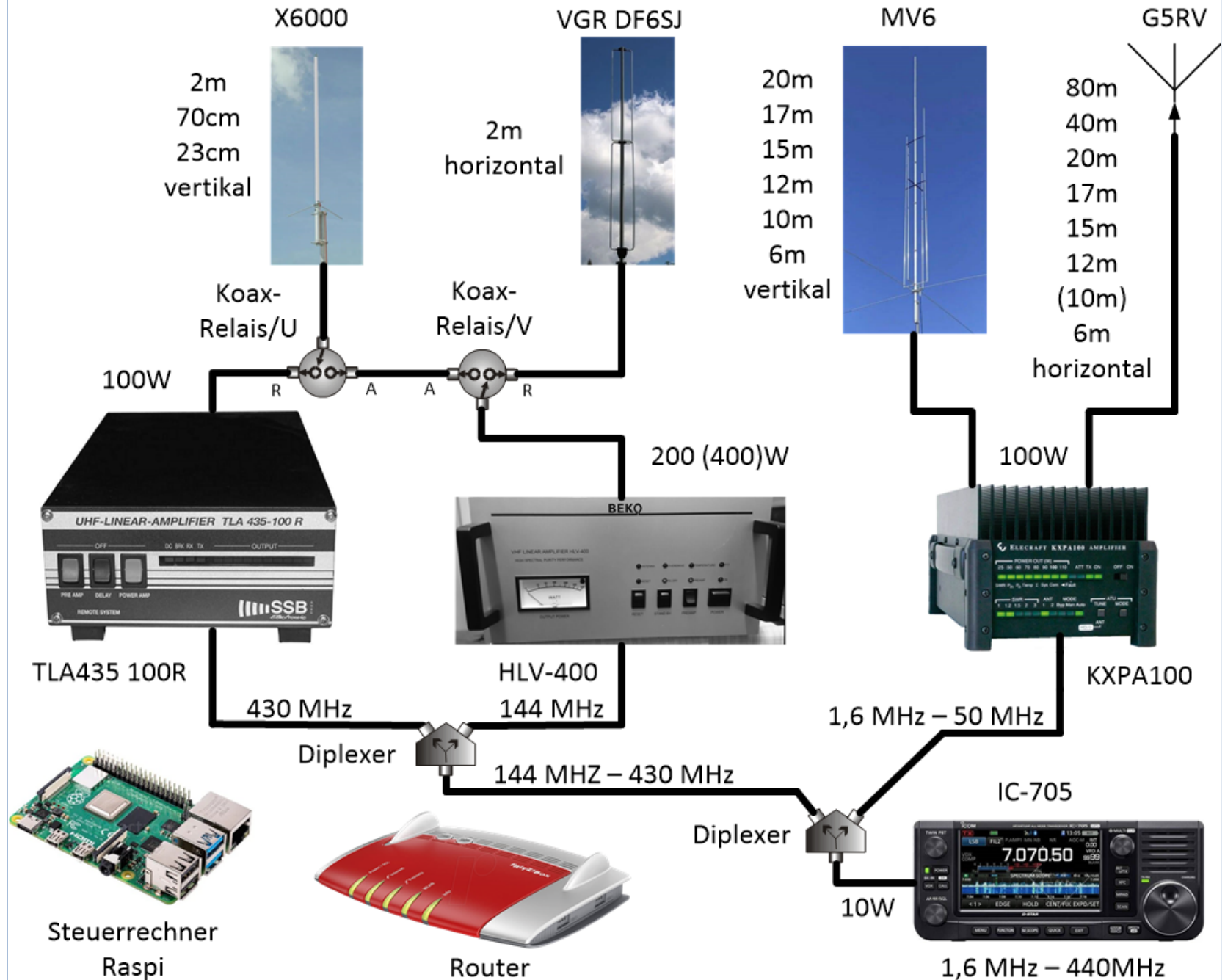


CAT – Steuerung an Beispielen

- 1. ICOM-705 als Remote-Station
 - Steuerung über RaspberryPi
- 2. YAESU FT-710 mit externem Tuner
 - 2.1 Steuerung über PC
 - 2.2 Steuerung über ESP32
- Verweise

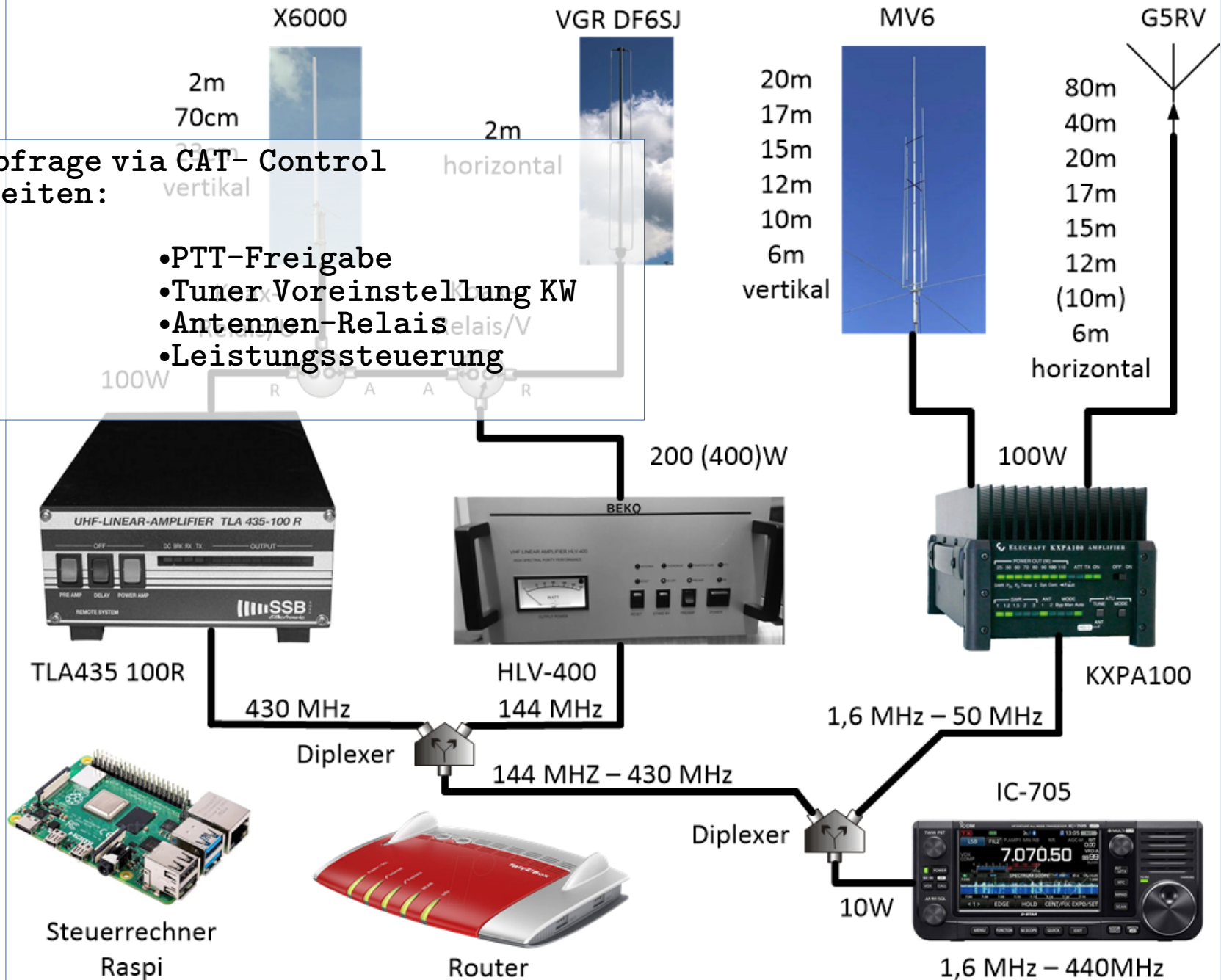
1. Remote- Station mit IC-705



Aufgabe der Steuerung

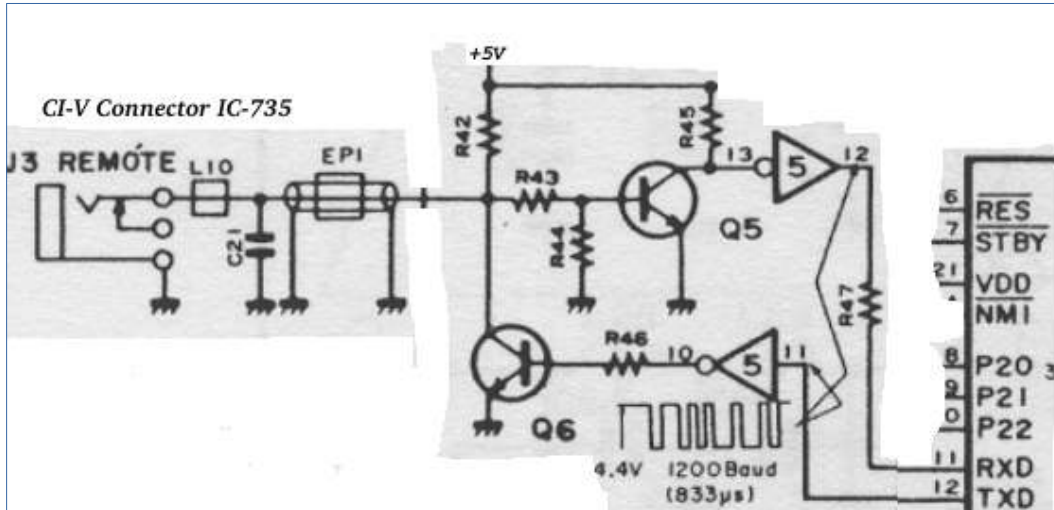
Frequenzabfrage via CAT- Control
Daraus ableiten:

- PTT-Freigabe
- Tuner Voreinstellung KW
- Antennen-Relais
- Leistungssteuerung

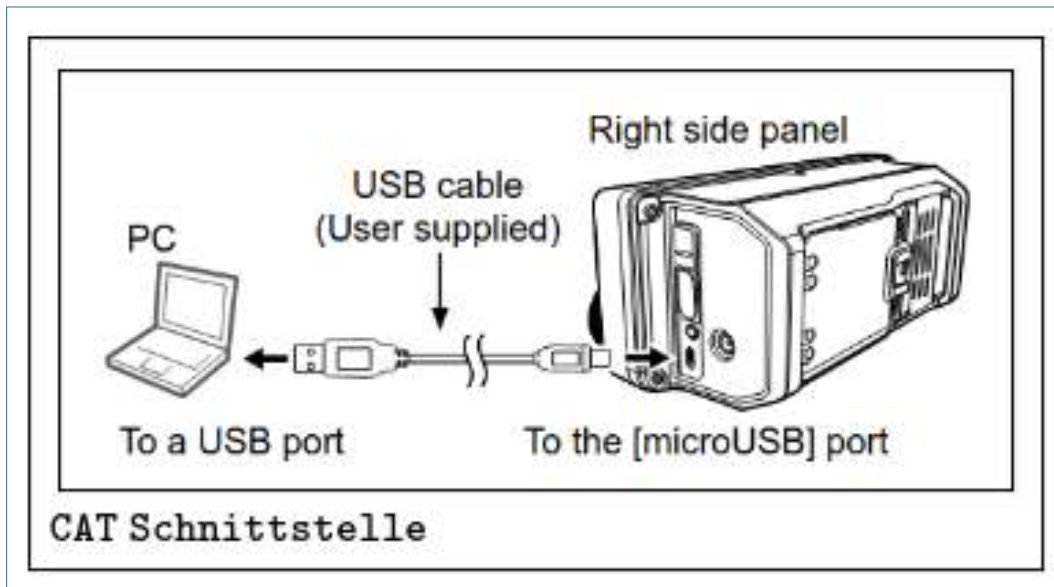


Hardware ICOM- CI-V (TTL)

CAT Steuerung = „Computer Aided Transceiver“



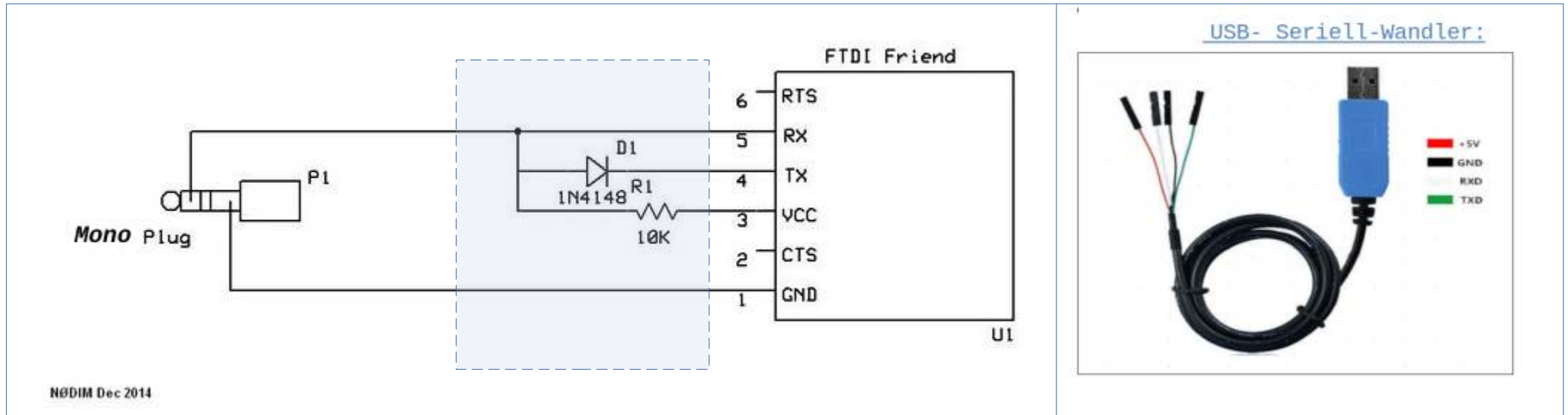
CI-V „Computer Interface V (Fünf)
TTL -Level (Ruhezustand ca +5V,
Aktivzustand ca. 0V)
Die Sende- und Empfangsleitung
(RXD und TXD) wird auf eine Leitung
kombiniert (Bus).



Beim IC-705 ist stattdessen eine virtuelle serielle Schnittstelle als **USB-Schnittstelle vorhanden**

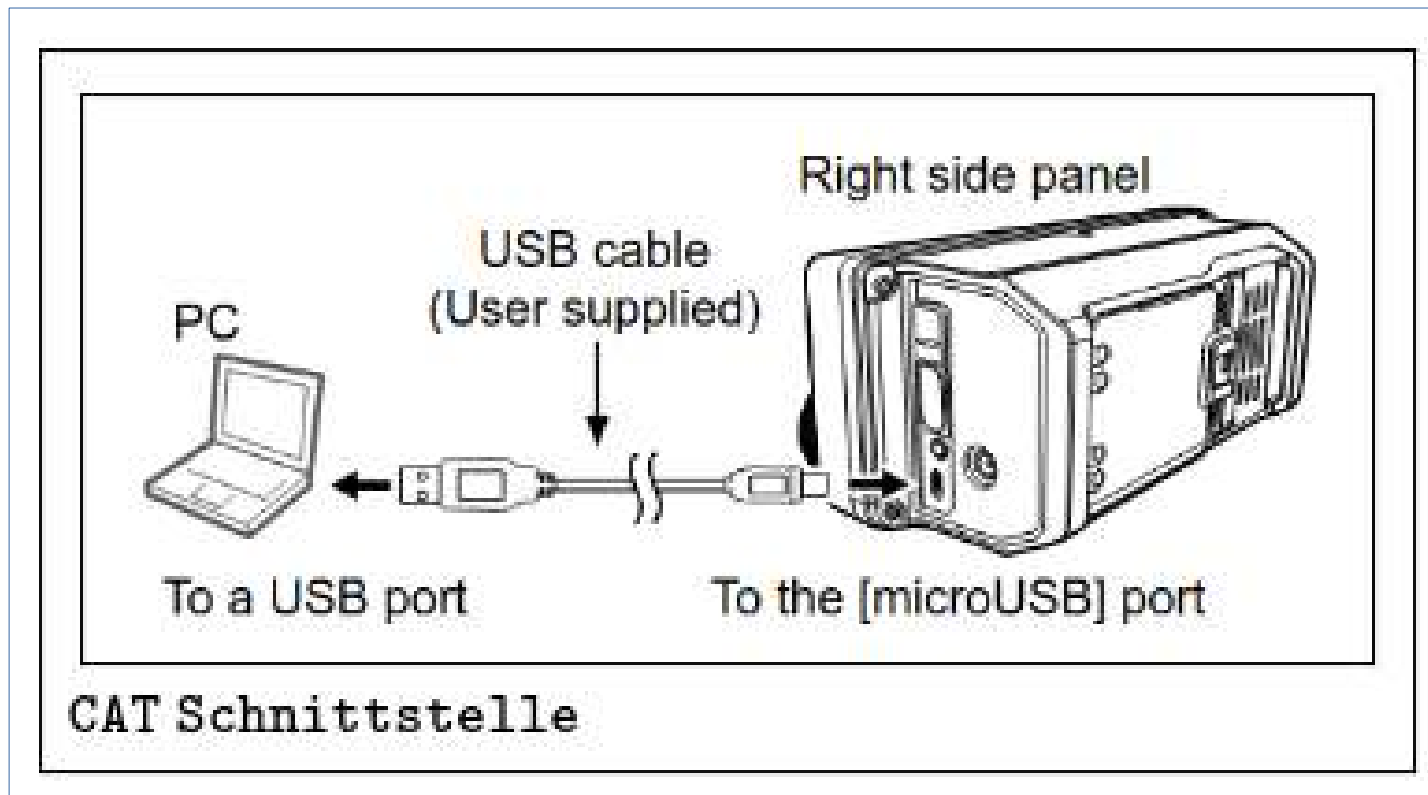
CI-V – USB-Adapter selbstgebaut

z.B. für IC-706
(bei IC-705 nicht erforderlich)



Adapter		CI-V Adapter
GND , schwarz	---	G
TXD , grün	---	rx
RXD , weiß	---	tx
+5v	---	:

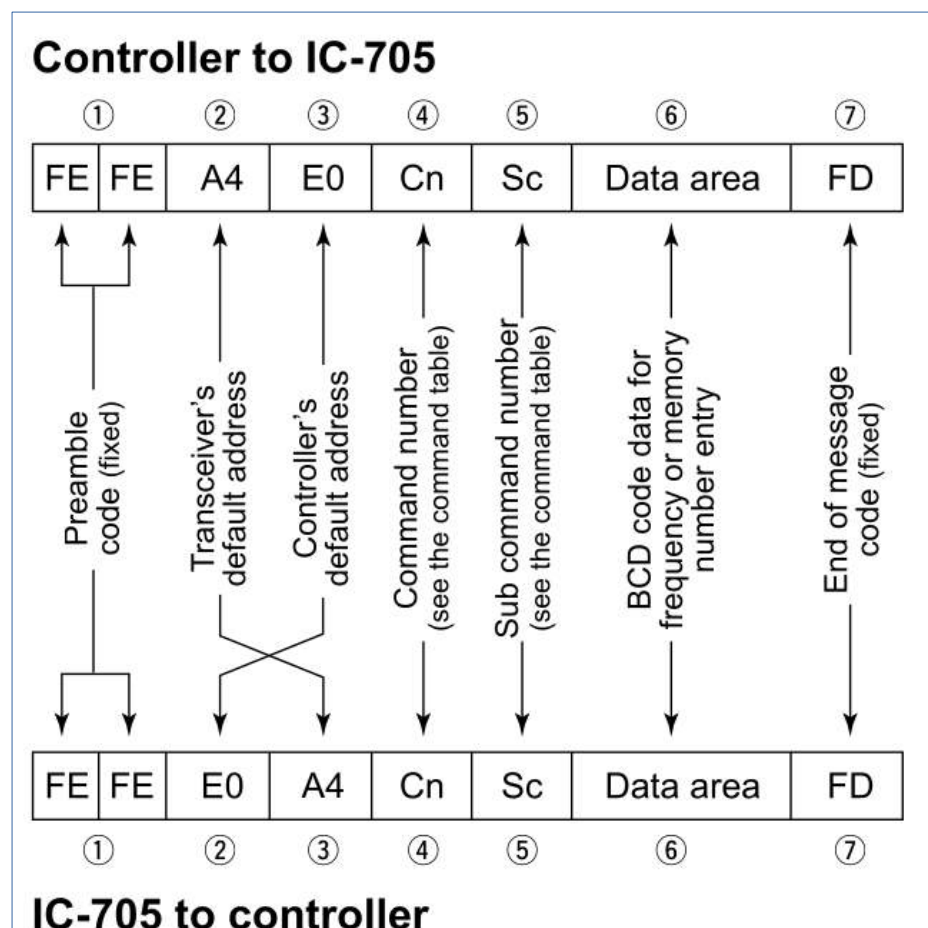
USB als virtueller COM-Port



```
pi@raspi62:~/app $ ls /dev/ttyA*  
/dev/ttyACM0 /dev/ttyACM1  
pi@raspi62:~/app $ ls /dev/ttyU*  
/dev/ttyUSB0
```



CI-V Command Table ICOM-705



◇ Command table

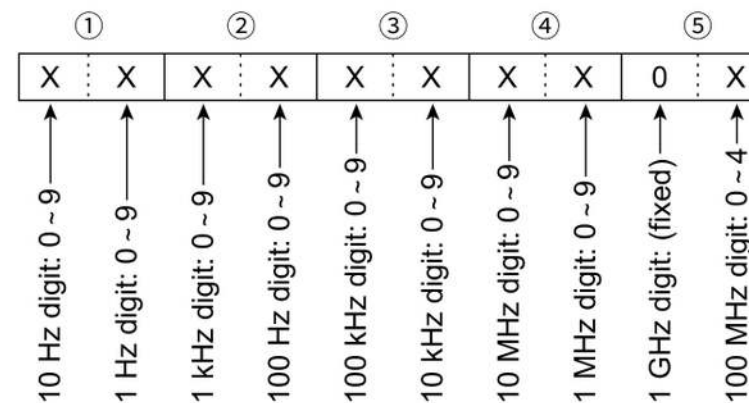
03*1	See p. 16.	Read the operating frequency
------	------------	------------------------------

Page 16:

◇ Command formats

• Operating frequency

Command: 00, 03, 05, 1C 03



Frequenzabfrage Datenformat

```
#ICOM Kommando zum Frequenz lesen:
# FE FE A4 E0 03 FD
# FE FE          <- kopf
# ..... A4      <- IC-705 CAT Adresse
# ..... E0       <- default ??
# ..... 03       <- Kommando Frequenz lesen
# ..... FD      <- Ende Zeichen
```

Interpretation der Antwort:

Antwort	= FE FE E0 A4 03 00 50 43 28 00 FD
Nutzdaten	= 00 50 43 28 00
Nutzdaten	= 00 28 43 50 00 (gedreht)
Roh-Ergebnis	= "0028435000" = text
Frequenz	= 28.435 MHz = float(text)/1e6

Frequenzabfrage ICOM Python Programm

[ic705_cat_mit kommentar.py]

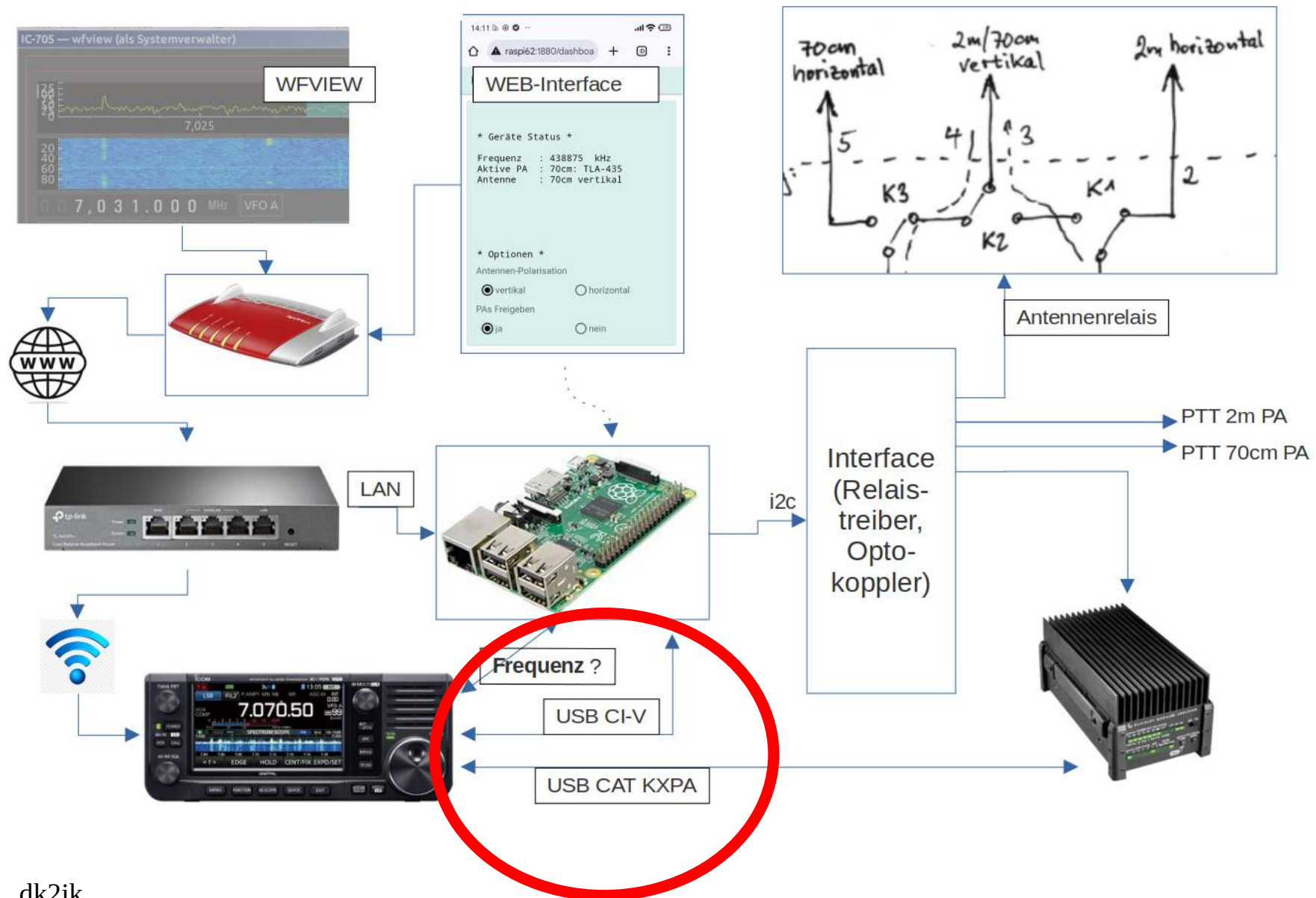
```
1
2 from serial import Serial    # modul serial laden
3
4                                     # Kommentar mit Beispieldaten...
5 def read_frequenz_IC_705():    # Frequenz lesen ...
6     kommando = [0xFE, 0xFE, 0xA4, 0xE0]+ [0x03] + [0xFD]
7                                     # FE FE A4 E0 03 FD
8     com = Serial('/dev/ttyACM0', 9600) # schnittstelle öffnen
9     com.write( bytes(kommando) )      # kommando schreiben
10    antwort = com.read_until(bytes([0xFD])) # FE FE E0 A4 03 00 50 12 07 00 FD
11    nutzdaten = antwort[5:9]          #          00 50 12 07 00
12    nutzdaten_rueckwaerts = nutzdaten[::-1] # (gedreht) 00 07 12 50 00
13    text = nutzdaten_rueckwaerts.hex() # text = "0007125000"
14    mhz = float(text)/1000_000         # mhz = float("0007125000")/1e6
15    com.close()                       # schnittstelle schliessen
16    return mhz                        # ergebnis = 7.125
17
18 if __name__ == '__main__':
19     frequenz = read_frequenz_IC_705() # frequenz lesen
20     print ( f' f={frequenz} MHz' )    # und anzeigen: f=7.125 MHz
```

Shell

```
>>> %Run 'ic705_cat_mit kommentar.py'
```

```
f=7.125 MHz
```

Wie hängt alles zusammen?



CAT-Control KXPA100

Befehlsformat

KXPA100-Befehle beginnen in der Regel mit einem Caret (^).
Alle Befehle verwenden den druckbaren ASCII-Zeichensatz.
Jeder serielle Befehl wird mit einem Semikolon abgeschlossen (;).

Hier wird verwendet:

^MT (ATU-Speicherabruf)

^MTffff; wobei ffff eine Frequenz in kHz ist.

**Der Tuner wird auf die Frequenz geschaltet
noch bevor er überhaupt HF sieht.**

über CAT-Schnittstelle
(einfaches ASCII-Format)

Befehl: ^MT7032;

== Memory Tune 7032 kHz

2. Tune Button für ext. Tuner am FT-710

(Tuner ist ein SG-230)

Möglichkeiten:

1. Manuell:

Mode=AM (ca. 25W) --> PTT am Mikrofon drücken --
>Tuner Arbeitet --> zurück auf alten ,Mode'

2. Verwendung der Buchse „REM/ALC“

Signal TX_REQ auf LOW --> TX sendet Träger
Leistung reduzieren mit negativen Spannung am
Signal ALC; -4.9V --> 10W. (TUN/LIN PORT SELECT
auf ,Linear')

3. CAT-Schnittstelle:

eine Kommandofolge senden, die den manuellen
Tune-Vorgang nachbildet.

Python Programm Frequenzabfrage YAESU

(Zum Vergleich: gleiche Aufgabe wie bei ICOM)

cat_pc_basic.py

```
12
13 port = '/dev/ttyUSB0'          # hier anpassen
14 baud = 38400
15
16 my_ser = serial.Serial( port, baud )
17
18 def cat( cmd ):
19     my_ser.write(cmd.encode())  # ascii in bytes umwandeln und senden
20     time.sleep(0.2)            # etwas warten
21     antwort = my_ser.read_all() # und antwort lesen
22     return antwort.decode()    # bytes in ascii umwandeln
23
```

Shell

```
>>> %Run cat_pc_basic.py
>>> cat('fa;')
'fa007030160;'
```

Experimente live

```
Shell >
>>> cat()           Frequenz ?
                        (default Argument)
'fa007029980;'

>>> cat('md0;')      Mode?
'md07;'

>>> cat('pc;')        Power?
'pc077;'

>>> cat('pc099;')     Power =99W
''

>>> cat('pc;')        Power ?
'pc099;'

>>> cat('fa007039000;') Frequenz = ...!
''

>>> cat()           Frequenz ?
'fa007039000;'
```

```
def cat( cmd = 'fa;'):
    my_ser.write(cmd.encode()) # ascii in bytes umwandeln und senden
    time.sleep(0.2)           # etwas warten
    antwort = my_ser.read_all() # und antwort lesen
    return antwort.decode()    # bytes in ascii umwandeln
```


2.1. Tune Programm am PC

Aufgabe: eine Kommandofolge senden, die den manuellen Tune-Vorgang nachbildet

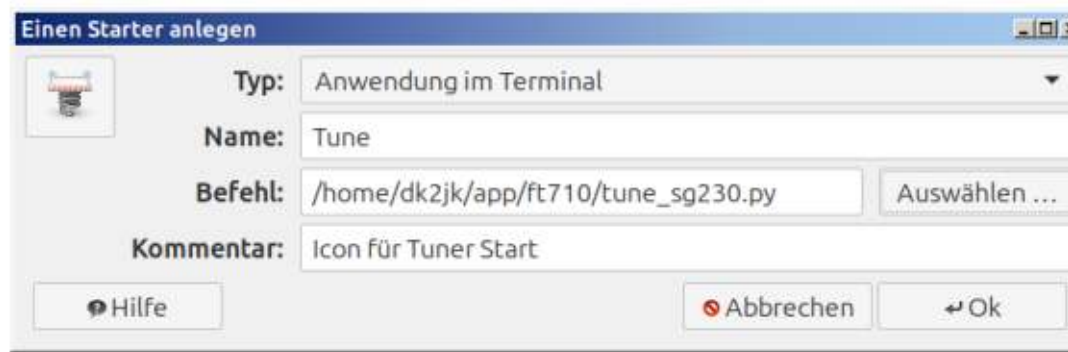
```
def tune_function():
    meter = get('MS;') # 'Meter'- Einstellung lesen
    mode = get('MD0;') # 'Mode' lesen
    power = get('PC;') # 'Power' lesen
    afgain= get('AG0;') # 'AF GAIN' lesen
    beep = get('EX030101;') # 'beep' lesen
    write('EX030101050;') # beep auf laut , falls aus
    write('AG0000;') # lautstärke aus
    write('MS50;') # 'Meter'- Einstellung auf 'SWR' stellen
    write('PC010;') # 'Power' auf 10 Watt schalten
    write('MD06;') # 'Mode' auf RTTY schalten
    write('MX1;') # 'PTT' einschalten
    sleep(tune_dauer) # x Sekunden lang Träger senden
    write('MX0;') # 'PTT' ausschalten
    write( mode ) # die gelesenen Werte wieder herstellen
    write( power )
    write( meter )
    write( afgain )
    write ( beep )
    close() # Schnittstelle schliessen
    return
```

Komplettes Programm im
Anhang

Tune Icon Link (Linux)

Link auf dem Linux Schreibtisch erstellen

Wählen Sie auf dem Schreibtisch mit der rechten Maustaste „Starter anlegen“ und tragen Sie unter ‚Befehl‘ den kompletten Pfad des Python-Programms ein.



Damit das Icon des „Starters“ besser aussieht, kann man im Starter-Text mit einem Texteditor folgende Zeile ergänzen: „Icon = python3“.

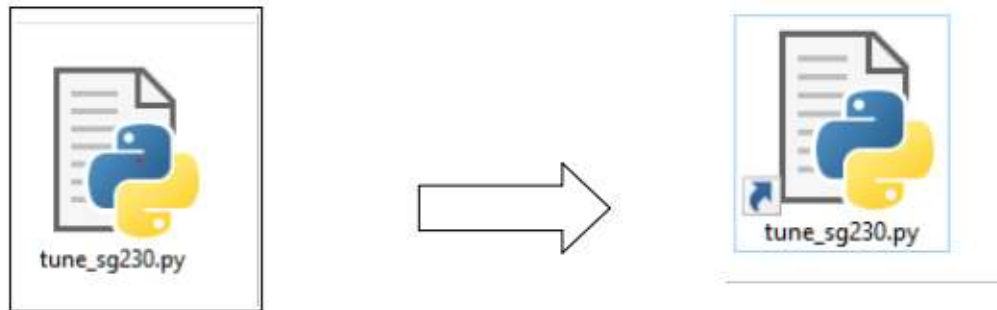
```
Tune.desktop *
1  #!/usr/bin/env xdg-open
2  [Desktop Entry]
3  Version=1.0
4  Type=Application
5  Terminal=true
6  Name[de_DE]=Tune
7  Icon = python3
8  Comment[de_DE]=Icon für Tuner Start
9  Exec=/home/dk2jk/app/ft710/tune_sg230.py
10 Name=Tune
11 Comment=Icon für Tuner Start
12
```



Tune Icon Link (Windows)

Link auf dem Windows Desktop erstellen

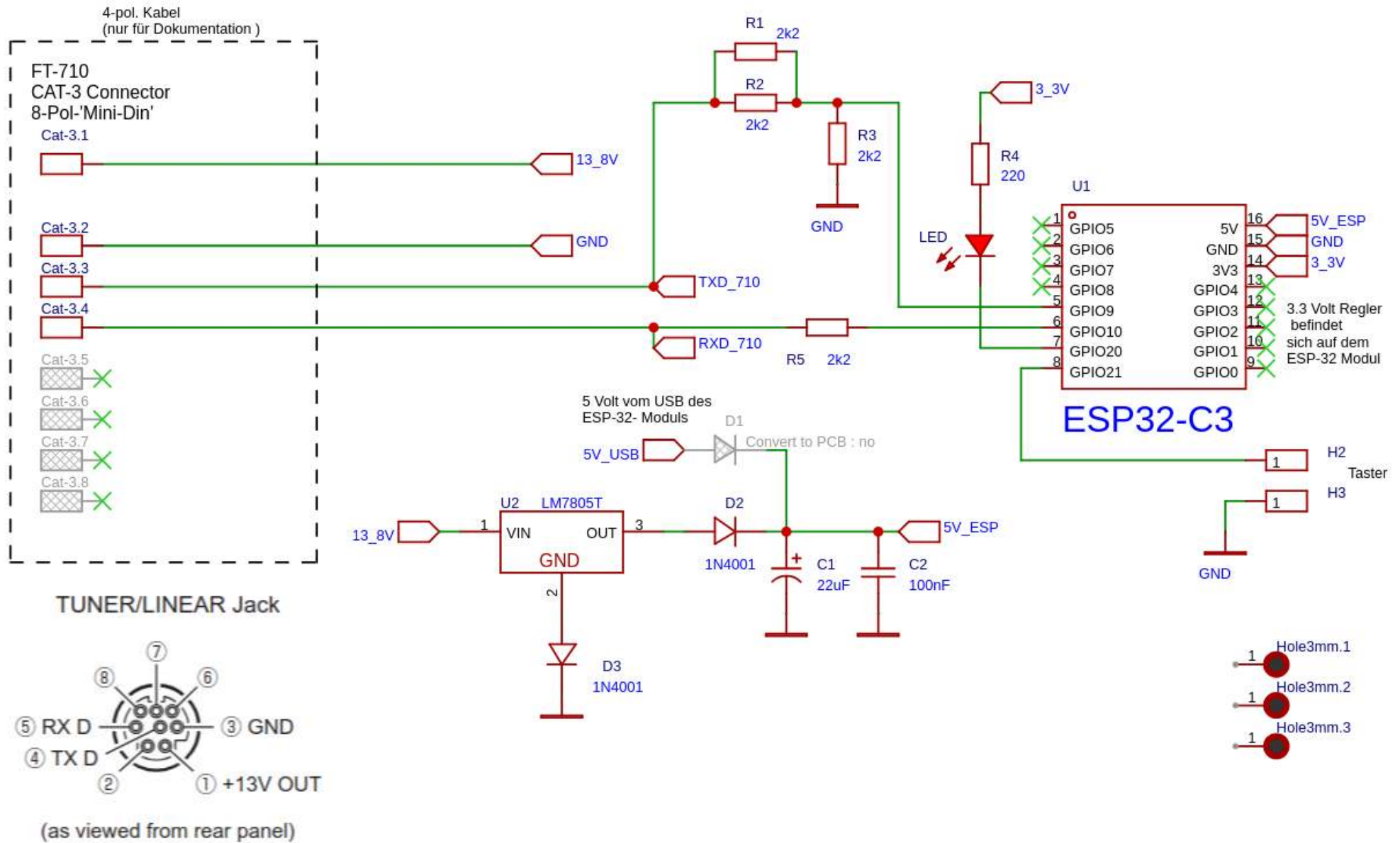
Wähle im Dateimanager das Programm aus und erstelle mit der rechten Maustaste eine Verknüpfung:



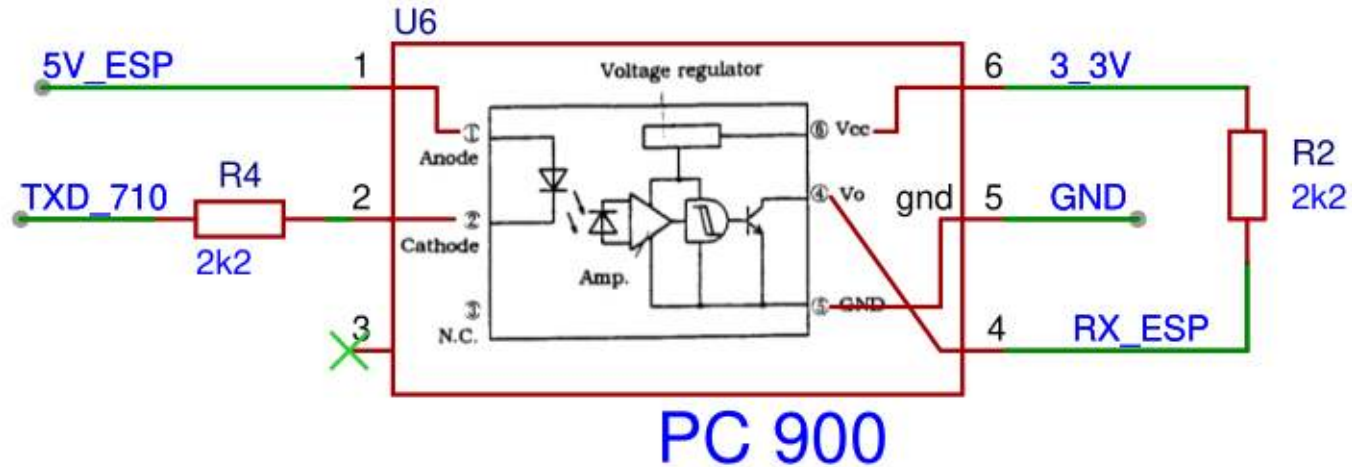
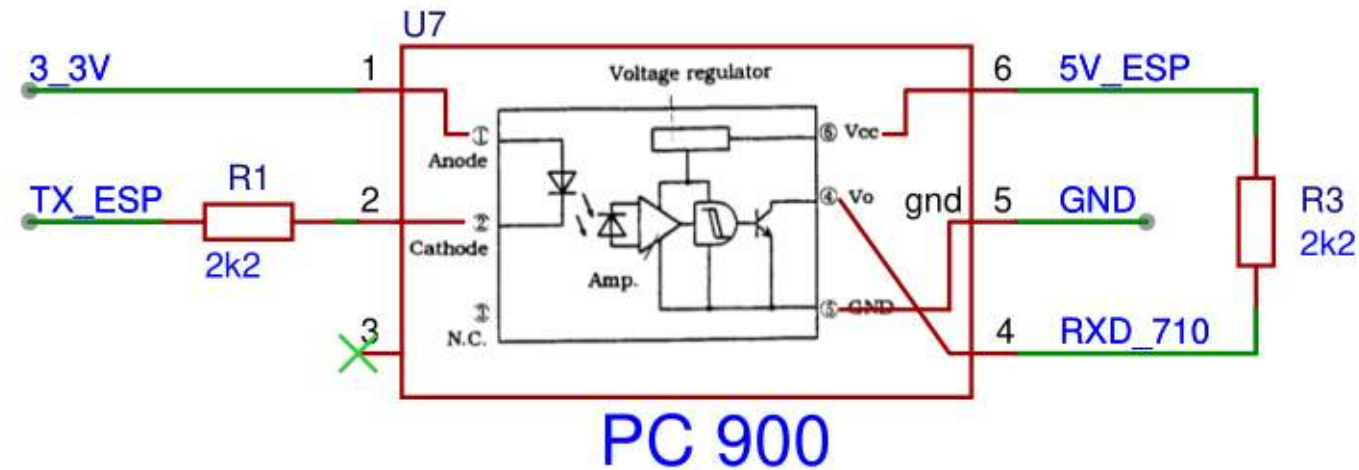
Die Verknüpfung wird als „Tuner-Start-Button“ auf den Desktop gezogen.

Beim Start über den Desktop öffnet sich ein Terminal- Fenster mit Statusanzeigen..

2.2. Tune Taste mit ESP-32



Variante mit Optokoppler



MicroPython Programm ESP32

```
def tune_function():
    meter = get('MS;') # 'Meter'- Einstellung lesen
    mode = get('MD0;') # 'Mode' lesen
    power = get('PC;') # 'Power' lesen
    afgain= get('AG0;') # 'AF GAIN' lesen
    beep = get('EX030101;') # 'beep' lesen
    write('EX030101050;') # beep auf laut , falls aus
    write('AG0000;') # lautstärke aus
    write('MS50;') # 'Meter'- Einstellung auf 'SWR' stellen
    write('PC010;') # 'Power' auf 10 Watt schalten
    write('MD06;') # 'Mode' auf RTTY schalten
    write('MX1;') # 'PTT' einschalten
    sleep(tune_dauer) # x Sekunden lang Träger senden
    write('MX0;') # 'PTT' ausschalten
    write( mode ) # die gelesenen Werte wieder herstellen
    write( power )
    write( meter )
    write( afgain )
    write( beep )
    close()
    return
```

esp32_tune.py

```
while True:
    x= taste() # taste lesen , low aktiv
    led(x) # led an solange taste gedrueckt ist
    if x ==0: # taste gedrueckt ?
        tune_function()
```


Verweise

tbd.

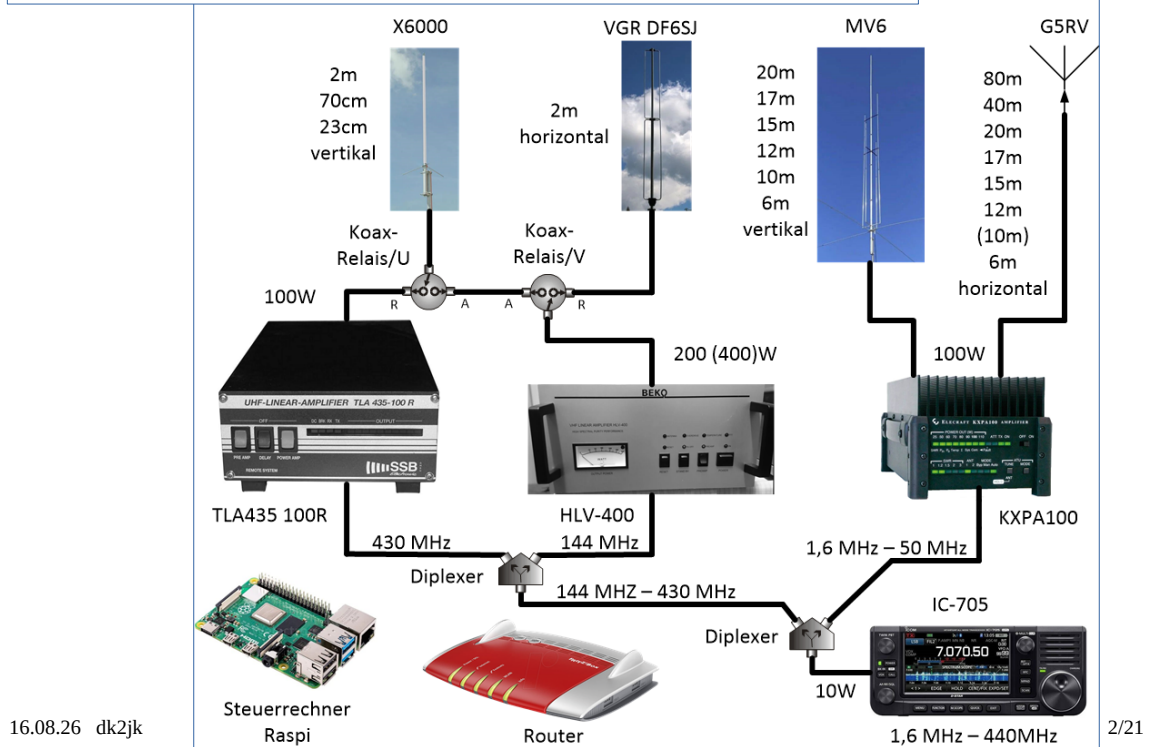
CAT – Steuerung an Beispielen



- 1. ICOM-705 als Remote-Station
 - Steuerung über RaspberryPi
- 2. YAESU FT-710 mit externem Tuner
 - 2.1 Steuerung über PC
 - 2.2 Steuerung über ESP32
- Verweise

Übersicht

1. Remote- Station mit IC-705



Hier die Ansicht der Remote- Station mit einem IC-705.

Der Transceiver wurde erweitert um je eine Endstufe für 70cm, 2m und KW.

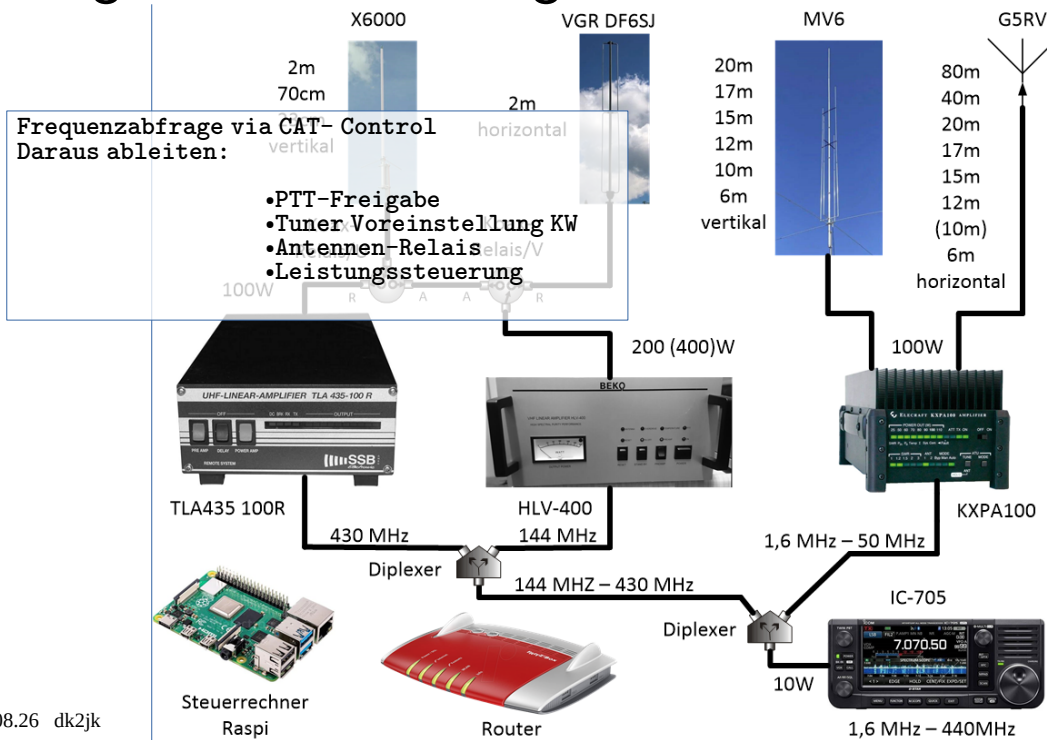
Ausserdem wurden mehrere fernsteuerbare Antennen vorgesehen.

Der Transceiver wird über die Remote-Software WFVIEW ferngesteuert.

Über WFVIEW ist jedoch die Zuschaltung individueller Endstufen und die Umschaltung der Antennen nicht möglich.

Hier kommt ein RaspberryPi ins Spiel, der über CAT- Steuerung die Frequenz abfragt.

Aufgabe der Steuerung



Abhängig von der Frequenz wird der PTT-Eingang der jeweiligen Endstufen freigegeben.

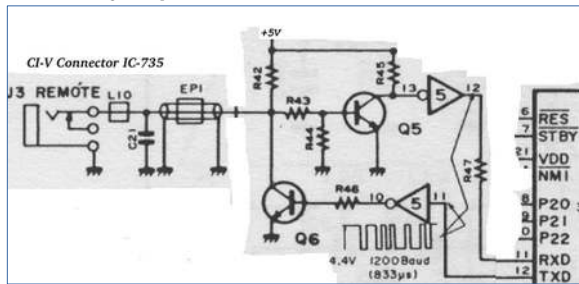
Der Tuner der Elecraft KXPA100 wird mit der aktuellen Frequenz voreingestellt, so dass der Tuner auch bei Empfang optimal arbeitet.

Weitere Steuermöglichkeiten- wie PA An oder Aus - erfolgen über ein zusätzliches WEB-Interface- was aber hier kein Thema sein soll - .

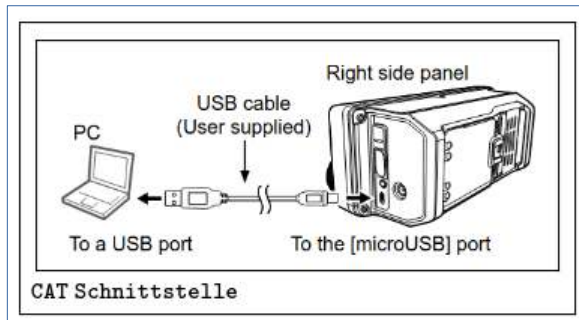
Beschäftigen wir uns nun mit der CAT-Steuerung des IC-705 ...

Hardware ICOM- CI-V (TTL)

CAT Steuerung = „Computer Aided Transceiver“



CI-V „Computer Interface V (Fünf)
TTL -Level (Ruhezustand ca +5V,
Aktivzustand ca. 0V)
Die Sende- und Empfangsleitung
(RXD und TXD) wird auf eine Leitung
kombiniert (Bus).



Beim IC-705 ist stattdessen eine
virtuelle serielle Schnittstelle
als **USB-Schnittstelle** vorhanden

16.08.26 dk2jk

4/21

Zunächst ein paar Worte zur CAT-Hardware.

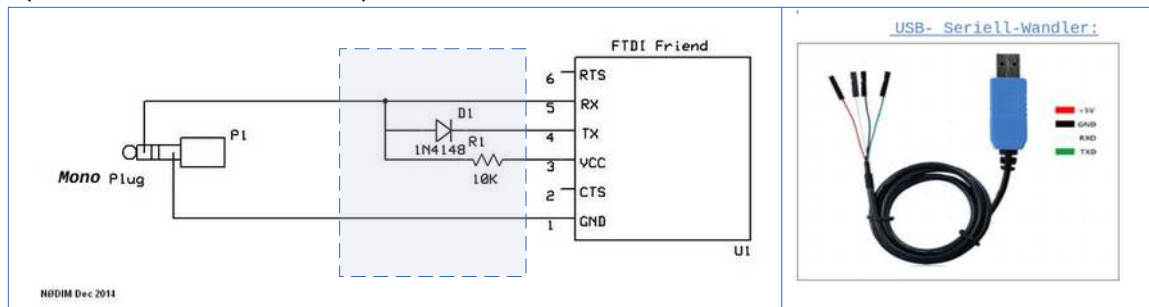
Grundsätzlich ist die CAT-Schnittstelle eine asynchrone serielle Schnittstelle mit Startbit , 8-Bit Daten, Stopbit. Die Baudrate wird im Transceiver eingestellt.

Bei ICOM heisst die Schnittstelle CI-V(fünf) . Ursprünglich ist es ein ‚Ein-Draht-Bus‘ mit TTL-Pegel. Es wird abwechselnd auf dem Bus gesendet oder empfangen; nur der Sender (TXD) zieht den Bus auf LOW. Das Schaltbild ist aus ein IC-735.

Beim IC-705 ist stattdessen eine virtuelle serielle Schnittstelle als USB-Schnittstelle vorhanden. Das ist bei den meisten modernen Transceivern der Fall. Die TTL- Schnittstelle ist dann oft noch extra vorhanden.

CI-V – USB-Adapter selbstgebaut

z.B. für IC-706
(bei IC-705 nicht erforderlich)

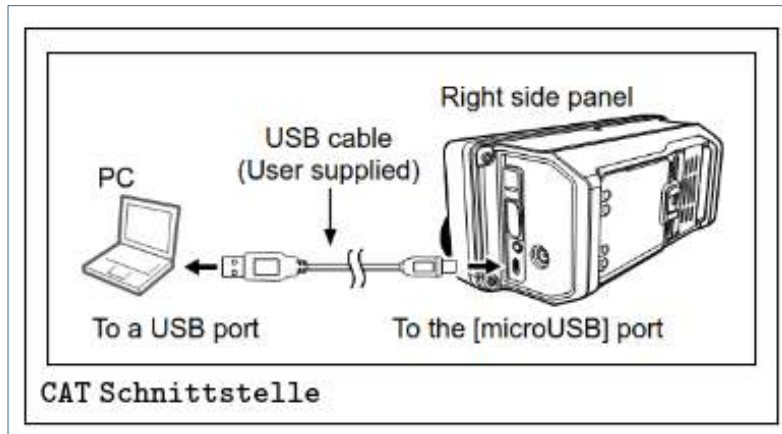


Adapter	---	CI-V Adapter
GND ,schwarz	---	G
TXD ,grün	---	rx
RXD ,weiß	---	tx
+5V	---	:

Für den Fall, dass eine USB-Schnittstelle benötigt wird, aber nur eine TTL-Schnittstelle vorhanden ist, so benötigt man einen USB-TTL-Adapter mit ein wenig zusätzlicher Hardware.

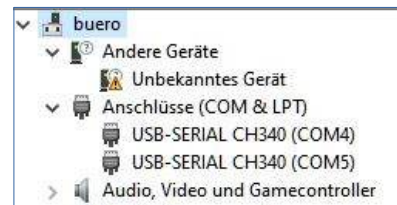
Auf diese Weise konnte ich einen IC-706 über die USB-Schnittstelle eines Raspi steuern, da mir zu Beginn des Projekts kein IC-705 zur Verfügung stand.

USB als virtueller COM-Port



```
pi@raspi62:~/app $ ls /dev/ttyA*  
/dev/ttyACM0 /dev/ttyACM1  
pi@raspi62:~/app $ ls /dev/ttyU*  
/dev/ttyUSB0
```

16.08.26 dk2jk



Beim Start des Projekts stellt sich oft die Frage: „Wie heisst die Schnittstelle“, mit der ich mich verbinden will.

Da der Raspi mit Linux läuft, schaue ich im Verzeichnis „/dev/“ nach Geräten mit dem Namen „ttyA*“ (das ist der IC-705) und nach Geräten mit dem Namen „ttyU*“ (das ist die KXPA100) .

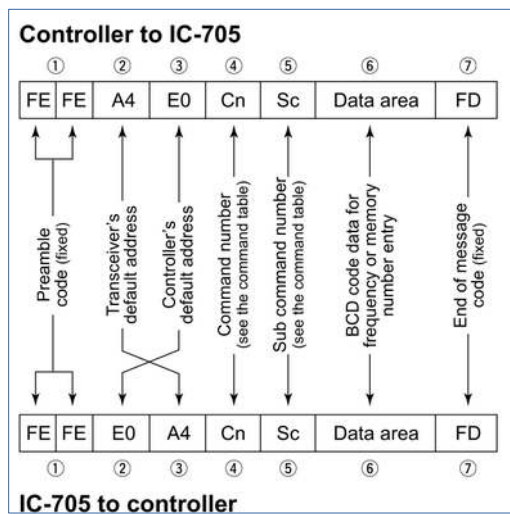
Auf einem Windows-PC sucht man die Schnittstellen-Namen im Gerätemanager unter „Anschlüsse (COM)“

Bei Verwendung eines USB-Hubs verändern sich die Gerätenamen.

Die im folgendenden vorgestellten Python-Programme für den Raspi laufen fast unverändert auf einem Linux- oder Windows-PC.

Man kann auf diese Weise Programmteile am PC ausprobieren und später auf den Remote-Raspi übertragen.

CI-V Command Table ICOM-705



◇ Command table

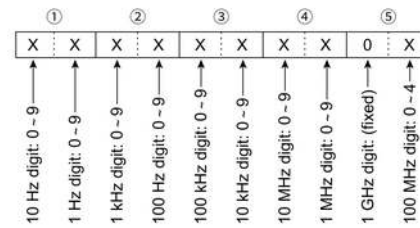
03*1	See p. 16.	Read the operating frequency
------	------------	------------------------------

Page 16:

◇ Command formats

• Operating frequency

Command: 00, 03, 05, 1C 03



16.08.26 dk2jk

Wir beschäftigen uns nun mit den CI-V- Kommandos und deren Codierung.

Bei ICOM werden die Daten hexadezimal kodiert; ein Hex- Buchstaben-Paar entspricht einem Byte.

Die damit verbundenen Werte als Zahl sind unwichtig; wir betrachten nur die HEX-Codes.

Frequenzabfrage Datenformat

```
#ICOM Kommando zum Frequenz lesen:
# FE FE A4 E0 03 FD
# FE FE                <- kopf
# ..... A4            <- IC-705 CAT Adresse
# ..... E0            <- default ??
# ..... 03            <- Kommando Frequenz lesen
# ..... FD            <- Ende Zeichen
```

Interpretation der Antwort:

```
Antwort      = FE FE E0 A4 03 00 50 43 28 00 FD
Nutzdaten    = 00 50 43 28 00
Nutzdaten    = 00 28 43 50 00 (gedreht)
Roh-Ergebnis = "0028435000" = text
Frequenz     = 28.435 MHz = float(text)/1e6
```

16.08.26 dk2jk

8/21

Das Datenformat CI-V besteht aus:

Präambel	FE FE	
CAT-Adresse	A4	(für IC-705)
Controller Adresse	E0	
Kommando	03	(Frequenz lesen)
Ende-Zeichen	FD	

Die Antwort besteht aus den Daten des Kommandos, wobei Sender und Empfänger vertauscht sind ; dann folgen 5 Bytes Nutzdaten; wenn man die Nutzdaten von hinten liest, so kann man an den HEX-Bytes schon die Frequenz lesen; das Ergebnis muss noch durch 1000000 geteilt werden, so hat man die Frequenz in MHz.

Bei dem Kommando „Frequenz setzen“ folgen dem Befehl 0x05 die Frequenz- Daten als 5 Bytes im Format wie bei „Frequenz lesen“.
„Frequenz setzen“ verwende ich jedoch hier nicht.

Frequenzabfrage ICOM Python Programm

```
[ ic705_cat_mit kommentar.py ]
1
2 from serial import Serial      # modul serial laden
3
4                                     # Kommentar mit Beispieldaten...
5 def read_frequenz_IC_705():      # Frequenz lesen ...
6     kommando = [0xFE, 0xFE, 0xA4, 0xE0]+ [0x03] + [0xFD]
7                                     # FE FE A4 E0 03 FD
8     com = Serial('/dev/ttyACM0', 9600) # schnittstelle öffnen
9     com.write( bytes(kommando) )      # kommando schreiben
10    antwort = com.read_until(bytes([0xFD])) # FE FE E0 A4 03 00 50 12 07 00 FD
11    nutzdaten = antwort[5:9]           #
12    nutzdaten_rueckwaerts = nutzdaten[::-1] # (gedreht) 00 07 12 50 00
13    text = nutzdaten_rueckwaerts.hex()  # text = "0007125000"
14    mhz = float(text)/1000_000          # mhz = float("0007125000")/1e6
15    com.close()                       # schnittstelle schliessen
16    return mhz                        # ergebnis = 7.125
17
18 if __name__ == '__main__':
19     frequenz = read_frequenz_IC_705() # frequenz lesen
20     print ( f' f={frequenz} MHz' )    # und anzeigen: f=7.125 MHz

Shell
>>> %Run 'ic705_cat_mit kommentar.py'
f=7.125 MHz
```

16.08.26

So sieht da Python Skript aus, mit dem die Frequenz gelsen wird.

Zeile 19: Funktion ,read_frequenz_IC705 wird aufgerufen, welche die Frequenz liefert.

Zeile 6: das Kommando wird zusammengesetzt(eine Liste mit Werten, hier in hex eingegeben.)

Zeile 8: Schnittstelle wir eröffnet; der Name der Schnittstelle wurde vorher im Gerätemanager abgelesen.

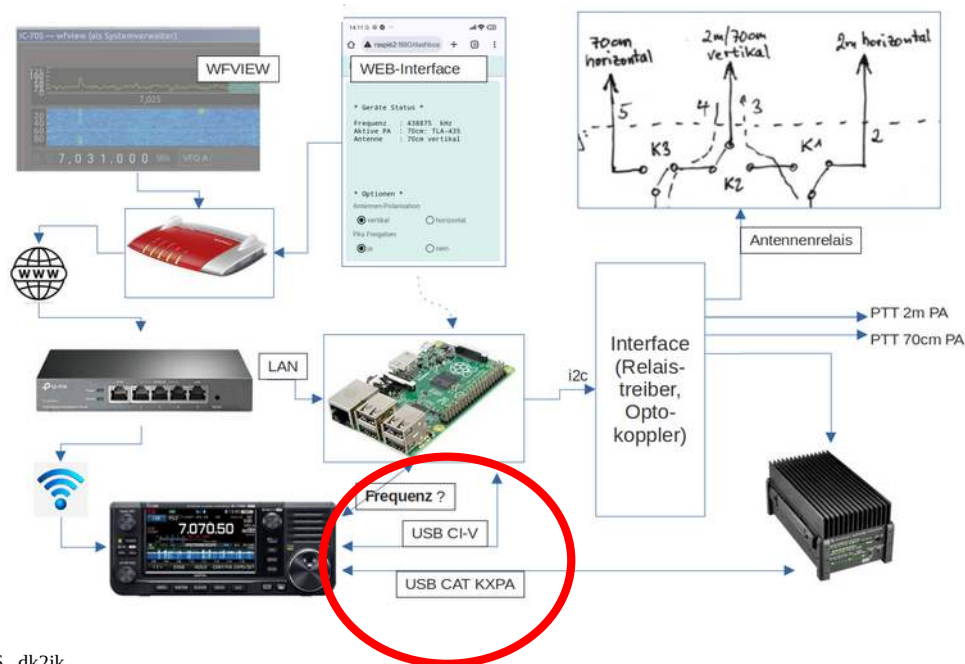
Zeile 9: Daten werden als Bytes geschrieben.

Zeile 10: Antwort wird gelesen (hier andere Frequenz als im vorigen Beispiel)

Zeile 11 ...13: Nutzdaten werden herausgefiltert und formatiert

Zeile 14: Ergebnis in MHz

Wie hängt alles zusammen?



16.08.26 dk2jk

10/21

Hier ist nochmals die ganze Hardware abgebildet. Die wesentliche Aufgabe des Raspi ist, die Frequenz über die CAT-Schnittstelle zu lesen. Abhängig von der eingestellten Frequenz werden über ein I2C Interface die passenden Antennen-Relais geschaltet. Ausserdem wird die PTT der zugehörigen PA freigegeben. Der Tuner in der KXPA100 erhält ebenfalls die Frequenz-Info, damit der Tuner auch bei Empfang schon richtig steht. Beim anschliessenden Senden stehen die Tuner-Relais schon in der richtigen Stellung; (für viele Frequenzen hat die KXPA100 die Tunerstellungen gespeichert)

Über ein zusätzlichen WEB-Interface ("Dashboard") wird die PA freigegeben und die Antenne bei UKW umgeschaltet werden von vertikaler auf horizontale Polarisierung.

Das WEB-Interface wird hier nicht näher betrachtet (Stichworte: NodeRED, MQTT)

CAT-Control KXPA100

Befehlsformat

KXPA100-Befehle beginnen in der Regel mit einem Caret (^).
Alle Befehle verwenden den druckbaren ASCII-Zeichensatz.
Jeder serielle Befehl wird mit einem Semikolon abgeschlossen (;).

Hier wird verwendet:

^MT (ATU-Speicherabruf)
^MTffff; wobei fffff eine Frequenz in kHz ist.

**Der Tuner wird auf die Frequenz geschaltet
noch bevor er überhaupt HF sieht.**

über CAT-Schnittstelle
(einfaches ASCII-Format)

Befehl: ^MT7032;
== Memory Tune 7032 kHz

16.08.26 dk2jk

Hier eine kurze Info , wie ein CAT- Befehl bei der KXPA100 aussieht.

Nebenbei:

Die CAT-Befehle des K2, K3 oder KX3 sehen ähnlich aus (Ohne (^)).

Das Schöne dabei ist, dass die Befehle in ASCII codiert sind, so dass man sie ohne Umwandlung als Text lesen kann.

Bei Kennwood und YAESU sind die Befehle ebenfalls in ASCII codiert.

2. Tune Button für ext. Tuner am FT-710

(Tuner ist ein SG-230)

Möglichkeiten:

1. Manuell:
Mode=AM (ca. 25W) --> PTT am Mikrofon drücken --
>Tuner Arbeitet --> zurück auf alten ,Mode'
2. Verwendung der Buchse „REM/ALC“
Signal TX_REQ auf LOW --> TX sendet Träger
Leistung reduzieren mit negativen Spannung am
Signal ALC; -4.9V --> 10W. (TUN/LIN PORT SELECT
auf ,Linear')
3. CAT-Schnittstelle:
eine Kommandofolge senden, die den manuellen
Tune-Vorgang nachbildet.

16.08.26 dk2jk

Es gibt 3 Möglichkeiten um einen Träger zum Abstimmen des Tuners zu senden.

Manuell: das ist mir zu umständlich

ALC: Funktioniert, man benötigt einen 3,5 mm Momo Stecker, einen Taster und eine negative Spannung, z.B. ein 9V Batterie plus Poti.

CAT: diese Schnittstelle will ich benutzen !!

Python Programm Frequenzabfrage YAESU

(Zum Vergleich: gleiche Aufgabe wie bei ICOM)

```
cat_pc_basic.py <
12
13 port = '/dev/ttyUSB0'      # hier anpassen
14 baud = 38400
15
16 my_ser = serial.Serial( port, baud )
17
18 def cat( cmd ):
19     my_ser.write(cmd.encode()) # ascii in bytes umwandeln und senden
20     time.sleep(0.2)           # etwas warten
21     antwort = my_ser.read_all() # und antwort lesen
22     return antwort.decode()   # bytes in ascii umwandeln
23

Shell <
>>> %Run cat_pc_basic.py
>>> cat('fa;')
'fa007030160;'
```

16.08.26 dk2jk

Bei YAESU sieht ein CAT- Befehl wesentlich einfacher aus als bei ICOM, da die Befehle ASCII-codiert sind. Dazu habe ich ein Python-Skript geschrieben mit der Funktion `cat(<befehl>)` mit dem Argument `<befehl>` als Übergabewert.

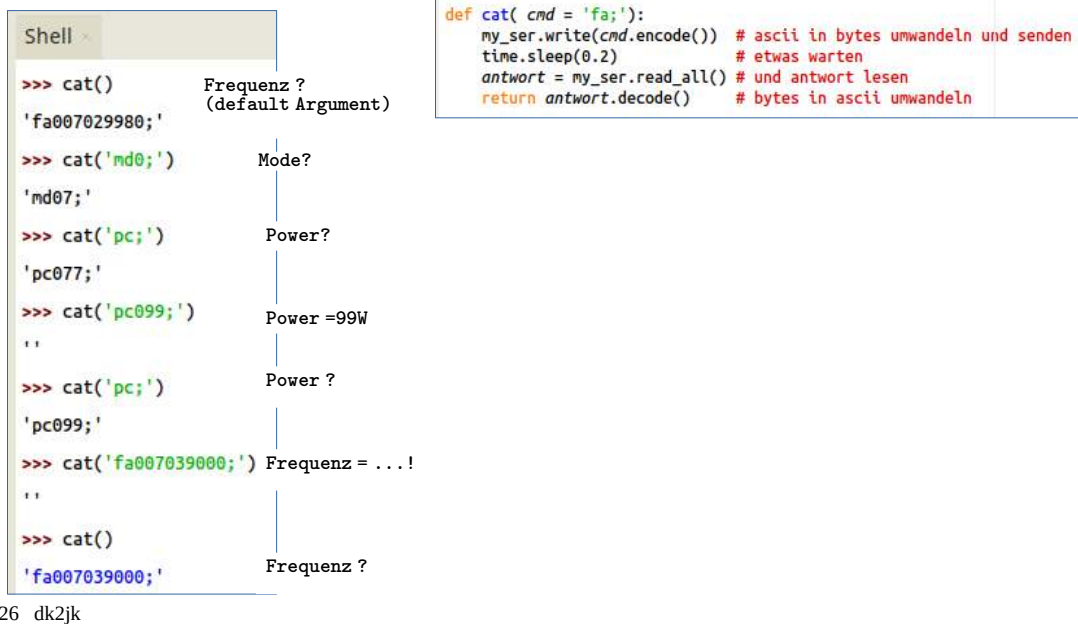
Im Beispiel wird die Frequenz mit dem Befehl `,fa;'` abgefragt.

Übrigens: CAT-Befehle dürfen grundsätzlich groß oder klein geschrieben werden.

Diese Funktion würde auch bei Elecraft K3,KX3 oder Kenwood funktionieren.

Kenwood habe ich nicht ausprobiert.

Experimente live



The screenshot shows a Python Shell window with a custom `cat()` function defined. The function takes a command string and sends it over a serial connection, waits for a response, and returns it. The shell shows the function being called with various commands like `'fa007029980;'`, `'md0;'`, `'pc;'`, and `'fa007039000;'`. The responses are displayed on the right side of the shell window.

```
def cat( cmd = 'fa;'):
    my_ser.write(cmd.encode()) # ascii in bytes umwandeln und senden
    time.sleep(0.2)           # etwas warten
    antwort = my_ser.read_all() # und antwort lesen
    return antwort.decode()    # bytes in ascii umwandeln
```

```
>>> cat()
'fa007029980;'
Frequenz ?
(default Argument)

>>> cat('md0;')
'md07;'
Mode?

>>> cat('pc;')
'pc077;'
Power?

>>> cat('pc099;')
'pc099;'
Power =99W

>>> cat('pc;')
'pc099;'
Power ?

>>> cat('fa007039000;')
'fa007039000;'
Frequenz = ...!

>>> cat()
'fa007039000;'
Frequenz ?
```

16.08.26 dk2jk

Hier habe ich die `cat()`-Funktion in der Python-Shell interaktiv ausprobiert.

Hier wird THONNY verwendet, eine Python IDE mit Dateimanager, Editor und Shell. Ist besonders attraktiv, da es wenige Einstellungen bedarf und in Verbindung mit allen Micropython- Prozessoren läuft.

Rückblende zum IC-705 Prjekt:

Ich habe Thonny IDE Remote über SSH verwendet, um die IC-705 Remote Station zu programmieren; d.h. Thonny läuft auf meinem Laptop, die Python Skripts laufen auf dem fernen Raspi.

2.1. Tune Programm am PC

Aufgabe: eine Kommandofolge senden, die den manuellen Tune-Vorgang nachbildet

```
def tune_function():
    meter = get('MS;') # 'Meter'- Einstellung lesen
    mode = get('MD0;') # 'Mode' lesen
    power = get('PC;') # 'Power' lesen
    afgain= get('AG0;') # 'AF GAIN' lesen
    beep = get('EX030101;') # 'beep' lesen
    write('EX030101050;') # beep auf laut , falls aus
    write('AG0000;') # lautstärke aus
    write('MS50;') # 'Meter'- Einstellung auf 'SWR' stellen
    write('PC010;') # 'Power' auf 10 Watt schalten
    write('MD06;') # 'Mode' auf RTTY schalten
    write('MX1;') # 'PTT' einschalten
    sleep(tune_dauer) # x Sekunden lang Träger senden
    write('MX0;') # 'PTT' ausschalten
    write( mode ) # die gelesenen Werte wieder herstellen
    write( power )
    write( meter )
    write( afgain )
    write ( beep )
    close() # Schnittstelle schliessen
    return
```

Komplettes Programm im
Anhang

16.08.26 dk2jk

Hier sind alle Befehle zusammengefasst für einen Tune-Button als Pseudocode. Die Funktionen `get()` und `write()` werden hier nicht gezeigt.

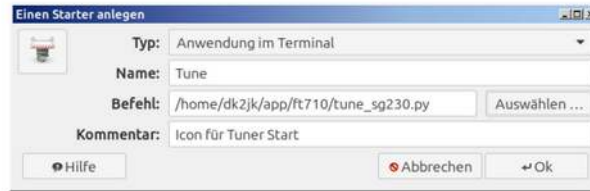
Im kompletten Programm sind noch einige Fehlermöglichkeiten abgefangen (z.b. Schnittstelle nicht vorhanden).

Zur Ansicht oder zum Testen des kompletten Programms steht ein Download zur Verfügung s. Anhang

Tune Icon Link (Linux)

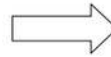
Link auf dem Linux Schreibtisch erstellen

Wählen Sie auf dem Schreibtisch mit der rechten Maustaste „Starter anlegen“ und tragen Sie unter ‚Befehl‘ den kompletten Pfad des Python-Programms ein.



Damit das Icon des „Starters“ besser aussieht, kann man im Starter-Text mit einem Texteditor folgende Zeile ergänzen: „Icon = python3“.

```
Tune.desktop *
1  #!/usr/bin/env xdg-open
2  [Desktop Entry]
3  Version=1.0
4  Type=Application
5  Terminal=true
6  Name[de_DE]=Tune
7  Icon = python3
8  Comment[de_DE]=Icon für Tuner Start
9  Exec=/home/dk2jk/app/ft710/tune_sg230.py
10 Name=Tune
11 Comment=Icon für Tuner Start
12
```



16.08.26 dk2jk

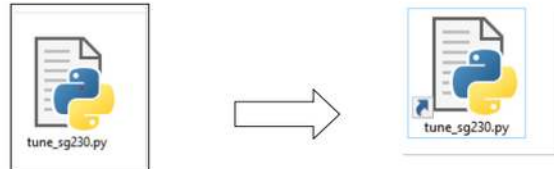
Da das Tune-Programm über einen Knopfdruck aufgerufen werden soll, benötigt man noch ein Icon als Starter.

Hier die Anleitung dazu.

Tune Icon Link (Windows)

Link auf dem Windows Desktop erstellen

Wähle im Dateimanager das Programm aus und erstelle mit der rechten Maustaste eine Verknüpfung:



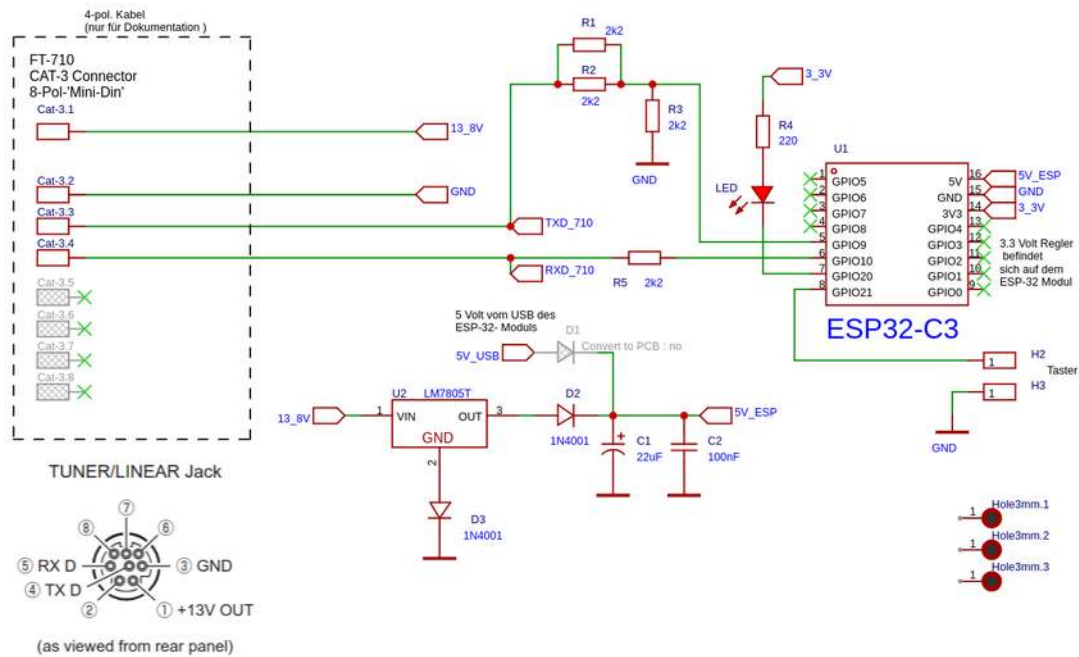
Die Verknüpfung wird als „Tuner-Start-Button“ auf den Desktop gezogen.

Beim Start über den Desktop öffnet sich ein Terminal- Fenster mit Statusanzeigen..

16.08.26 dk2jk

Hier die Anleitung zum Erstellen eines Links bei Windows.

2.2. Tune Taste mit ESP-32



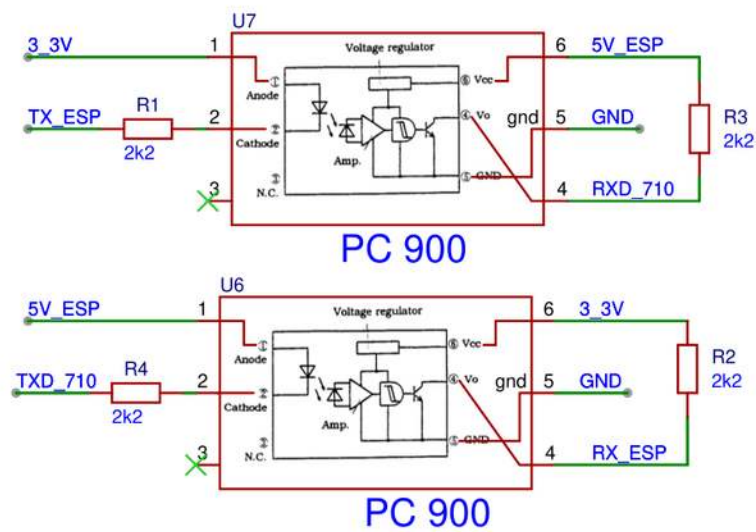
16.08.26 dk2jk

Python läuft als Micropython in vielen 32-bit-Prozessoren. Dazu wird in den Prozessor zunächst ein Micropython -Image geladen. Die Kommunikation geht nur über die serielle Schnittstelle (siehe Anhang „Micropython“)

Da hier der Prozessor mit 3.3 Volt arbeitet, muss für RXD des Prozessors ein Spannungsteiler 5v/ 3,3V vorgeschaltet werden. Der Sender TXD des Prozessors ist ok, da 3,3V als High-Pegel am RXD Eingang des Transceivers als TTL-HIGH akzeptiert wird.

Stromversorgung kommt vom Transceiver über einen 5V Spannungsregler; auf dem ESP32 Modul ist noch ein 3.3V Regler, da das Modul auch von USB versorgt werden kann.

Variante mit Optokoppler



16.08.26 dk2jk

Hier eine Pegelwandler 3.3v/5V als Variante mit Optokoppler.
Das soll nur der Vollständigkeit halber gezeigt werden.

MicroPython Programm ESP32

```
def tune_function():
    meter = get('MS;') # 'Meter'- Einstellung lesen
    mode = get('MD0;') # 'Mode' lesen
    power = get('PC;') # 'Power' lesen
    afgain= get('AG0;') # 'AF GAIN' lesen
    beep = get('EX030101;') # 'beep' lesen
    write('EX030101050;') # beep auf laut , falls aus
    write('AG0000;') # lautstärke aus
    write('MS50;') # 'Meter'- Einstellung auf 'SWR' stellen
    write('PC010;') # 'Power' auf 10 Watt schalten
    write('MD06;') # 'Mode' auf RTTY schalten
    write('MX1;') # 'PTT' einschalten
    sleep(tune_dauer) # x Sekunden lang Träger senden
    write('MX0;') # 'PTT' ausschalten
    write( mode ) # die gelesenen Werte wieder herstellen
    write( power )
    write( meter )
    write( afgain )
    write( beep )
    close()
    return
```

```
main.py*
#main.py
import esp32_tune
```

```
esp32_tune.py
while True:
    x= taste() # taste lesen , low aktiv
    led(x) # led an solange taste gedrueckt ist
    if x ==0: # taste gedrueckt ?
        tune_function()
```

16.08.26 dk2jk

Hier sind nochmals alle Befehle zusammengefasst für einen Tune-Button. Die Funktion selbst ist identisch mit der PC-Version.

Nur die Funktionen get() und write() sehen für Micropython etwas anders aus.

Es wird noch ein Tune Button benötigt (taste)

Das Programm läuft in einer Schleife und fragt den Taster ab.

Damit das Programm beim einschalten automatisch starten, muß in der Datei ,Main.py' das Tune Programm importiert werden ,siehe Ausschnitt. Auf diese Weise wird in Micropython ein Autostart gemacht.

Programm zum Download im Anhang

Verweise

tbd.